

Reto 2

Unidades de trabajo:

Programación de Inteligencia Artificial

Sistemas de Big Data

Big Data Aplicado

Modelos de Inteligencia Artificial

Reto 2: Cloud-Vision - Transformando Video en Datos

El Reto

Contexto: Vivimos en una era dominada por los datos visuales. Desde cámaras de seguridad en ciudades inteligentes y drones en agricultura, hasta control de calidad en líneas de producción y análisis de comportamiento en retail, las organizaciones generan miles de horas de video diariamente. Sin embargo, la gran mayoría de esta información es "oscura": datos no estructurados que requieren intervención humana para ser interpretados.

El verdadero desafío actual no es capturar las imágenes, sino procesarlas a escala para extraer valor de negocio en tiempo real. Las empresas necesitan sistemas capaces de "ver" y catalogar eventos automáticamente para tomar decisiones basadas en datos, sin depender de la revisión manual.

La imagen de la figura muestra cómo funciona el sistema en Prime video donde es capaz de identificar en tiempo real los actores que aparecen en pantalla.

Un posible reto sería desarrollar un sistema que analice tráilers de películas, identificando no sólo actores famosos y ciertos objetos clave para mejorar la experiencia de usuario y personalizar recomendaciones, por ejemplo, para a partir de esos datos, poder comprar ropa que salga en una serie.

Ejemplo:

Amazon busca responder preguntas clave que le permitan entender mejor qué información está presente en cada tráiler y cómo utilizarla para personalizar recomendaciones o realizar análisis posteriores. Todo esto está explicado en un documento que os ha mandado Amazon con el nombre "X-ray-Datos_uso".

*Si vosotros escogéis la temática, una vez escogida, se tendrán que proponer esas preguntas clave que definan nuestra estructura de tabla.

Objetivos / Resultados de Aprendizaje / ODS

Objetivos del reto:

Programación de Inteligencia Artificial

1. **Desplegar pipelines de procesamiento de imágenes en entornos cloud y/o on premise**, evaluando diferentes estrategias para su implementación eficiente.
2. **Gestionar recursos en la nube**, incluyendo contenedores, máquinas virtuales, balanceadores de carga y APIs, asegurando su mantenimiento, control adecuado y recurso monetario empleado.
3. **Analizar alternativas en entornos cloud**, comparando soluciones en términos de costo, rendimiento y escalabilidad para elegir la más adecuada.
4. **Utilizar APIs y servicios cloud de inteligencia artificial**, integrándose en el procesamiento de imágenes para mejorar la automatización.
5. **Configurar entornos locales de desarrollo, aplicando** buenas prácticas para la creación, prueba y despliegue de pipelines de imágenes.

Big Data Aplicado

1. Gestión de almacenamiento en S3.
2. Procesamiento y almacenamiento en DynamoDB.
3. Monitorización del sistema.
4. Optimización de las Read y Write capacity Units.
5. Gestión de permisos y roles.

Modelos de Inteligencia Artificial

1. Aplicar los conocimientos de modelos de visión.
2. Aplicar los conocimientos de sistemas expertos y robotizados.
3. Diseñar un sistema de visión artificial que utilice **YOLO (You Only Look Once)** para tareas de visión.
4. Evaluar y ajustar los parámetros del modelo YOLO para mejorar la precisión y eficiencia del sistema.
5. Integrar los modelos de IA con servicios de AWS.

Sistemas de Big Data

1. Gestionar la transmisión de flujos de datos en tiempo real implementando **Apache Kafka** como sistema de mensajería.
2. **Orquestrar flujos de trabajo (Data Pipelines)** mediante **Apache Airflow**.

TRANSVERSALES

- Competencia Digital
- Comunicación

OBJETIVOS DE DESARROLLO SOSTENIBLE



Resultados de Aprendizaje:

5071. Modelos de Inteligencia Artificial.

RA2. Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.

Criterios de evaluación:

- a) Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas.
- c) Se han caracterizado los modelos de automatización de tareas.
- f) Se han caracterizado los modelos de sistemas de Visión Artificial.
- g) Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

RA4. Analiza sistemas robotizados evaluando opciones de diseño e implementación.

Criterios de evaluación:

- b) Se han buscado soluciones a los problemas de los robots.
- d) Se han evaluado diferentes opciones en el diseño e implementación de sistemas robotizados.

RA5. Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.

Criterios de evaluación:

- b) Se han determinado las destrezas necesarias para representar y simular comportamientos básicos de sistemas de muy diversos ámbitos.
- d) Se han desarrollado estrategias de control definiendo los objetivos y las especificaciones de la respuesta del sistema.
- e) Se han relacionado los controladores inteligentes con el comportamiento del sistema.

RA6: Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.

Criterios de evaluación:

- a) Se han argumentado los posibles riesgos legales y éticos de la aplicación de Inteligencia Artificial.
- b) Se ha reconocido la necesidad de respetar la privacidad de los datos.
- c) Se ha decidido el cumplimiento estricto de la legalidad en su aplicación.
- d) Se ha integrado como parte del proceso la protección frente a previsibles errores y ataques (security by design).
- e) Se ha comprobado que se cumplen todas las normas legales y éticas en todas las áreas de la Inteligencia Artificial (privacy by design).
- f) Se han identificado y corregido los posibles sesgos de género en el desarrollo y aplicaciones de Inteligencia Artificial y Big Data.

5073. Programación de sistemas de inteligencia artificial

RA1: Caracteriza lenguajes de programación valorando su idoneidad en el desarrollo de Inteligencia Artificial.

- a. Principales características en un lenguaje de programación para Inteligencia Artificial. Bibliotecas. Rendimiento en ejecución. Herramientas. Soporte.
- b. Principales lenguajes de programación para Inteligencia Artificial: Python, R, Java, Javascript, NodeJS, JSON, entre otros.
- c. Programa informático. Etapas. Fundamentos del lenguaje de programación.

RA4: Evalúa modelos de automatización industrial y de negocio relacionándolos con los resultados esperados por las empresas

- Modelos de automatización. Tendencias.

RA2: Desarrolla aplicaciones de Inteligencia artificial utilizando entornos de modelado.

- Módulos predefinidos

5075. Big Data Aplicado

RA1: Gestiona soluciones a problemas propuestos, utilizando sistemas de almacenamiento y herramientas asociadas al centro de datos.

Criterios de evaluación:

- a. Se ha caracterizado el proceso de diseño y construcción de soluciones en sistemas de almacenamiento de datos.
- b. Se han determinado los procedimientos y mecanismos para la ingesta de datos.
- c. Se ha determinado el formato de datos adecuado para el almacenamiento.
- d. Se han procesado los datos almacenados.

- e. Se han presentado los resultados y las soluciones al cliente final en una forma fácil de interpretar.

RA2. Gestiona sistemas de almacenamiento y el amplio ecosistema alrededor de ellos, facilitando el procesamiento de grandes cantidades de datos, sin fallos y de forma rápida.

Criterios de evaluación:

- a. Se ha determinado la importancia de los sistemas de almacenamiento para depositar y procesar grandes cantidades de cualquier tipo de datos rápidamente.
- b. Se ha comprobado el poder de procesamiento de su modelo de computación distribuida.
- c. Se ha probado la tolerancia a fallos de los sistemas.
- d. Se ha determinado que se pueden almacenar tantos datos como se desee y decidir cómo utilizarlos más tarde.
- e. Se ha visualizado que el sistema puede crecer fácilmente añadiendo módulos.

RA4. Realiza el seguimiento de la monitorización de un sistema, asegurando la fiabilidad y estabilidad de los servicios que se proveen.

Criterios de evaluación:

- a. Se han aplicado herramientas de monitorización eficiente de los recursos.
- b. Se han recogido métricas, procesamiento y visualización de los datos.
- c. Se han generado alertas para detectar un riesgo o mal funcionamiento.
- d. Se ha comprobado que las herramientas usadas ofrecen un rendimiento elevado con rapidez.
- e. Se ha comprobado la fiabilidad de los datos según respuestas.
- f. Se ha analizado la estabilidad de servicios.

5074. Sistemas de Big Data

RA1. Aplica técnicas de adquisición y análisis de datos que integran, procesan y analizan la información, adaptando e implementando sistemas que las utilicen.

Criterios de evaluación:

- b) Se ha extraído de forma automática información y conocimiento a partir de grandes volúmenes de datos.
- c) Se han combinado diferentes fuentes y tipos de datos.
- d) Se ha construido un conjunto de datos complejos y se han relacionado entre sí.
- e) Se han establecido objetivos y prioridades, secuenciación y organización del tiempo de realización.
- f) Se han seleccionado e integrado sistemas de información que satisfacen las necesidades del problema.
- g) Se han determinado criterios de coste y calidad necesarios para la eficacia y eficiencia de la implementación de un sistema Big Data.

RA3. Gestiona y almacena datos facilitando la búsqueda de respuestas en grandes conjuntos de datos.

Criterios de evaluación:

- a) Se han extraído y almacenado datos de diversas fuentes para ser tratados en distintos escenarios.
- b) Se ha fijado el objetivo de extraer valor de los datos para lo que es necesario contar con tecnologías eficientes.
- c) Se ha comprobado que la revolución digital exige poder almacenar y procesar ingentes cantidades de datos de distinto tipo y descubrir su valor.
- d) Se han desarrollado sistemas de gestión, almacenamiento y procesamiento de grandes volúmenes de datos de manera eficiente y segura, teniendo en cuenta la normativa existente.
- e) Se han utilizado habilidades científicas en entornos de trabajo multidisciplinares.

1. Tareas a realizar

Tareas:

Sistemas de Big Data

1. **Ingesta y Streaming de Datos en Tiempo Real (Apache Kafka):**
 - a. Diseñar la arquitectura de mensajería: Configurar un clúster de Kafka y definir los tópicos necesarios para separar los diferentes flujos de datos (ej: raw_detections, processed_alerts).
 - b. Implementar Productores (Producers): Integrar en el script de detección (Python/YOLO) un productor de Kafka que envíe los resultados de cada frame (clase detectada, confianza, bounding box) al tópico correspondiente en formato JSON.
 - c. Implementar Consumidores (Consumers): Desarrollar un servicio consumidor que lea los mensajes del bus de Kafka en tiempo real y se encargue de la persistencia de los datos en la base de datos (DynamoDB) o de generar alertas inmediatas.
2. **Orquestación del Flujo de Trabajo (Apache Airflow):**
 - a. **Diseñar y programar DAGs:** Crear DAGs para gestionar tareas que sean cíclicas.
 - b. **Automatización:** Automatizar el lanzamiento del DAG, por ejemplo detectando la llegada de nuevos datasets.
 - c. **Gestión de tareas:** Configurar el pipeline para manejar reintentos en caso de fallo y enviar notificaciones de finalización o error del proceso.

Modelos de Inteligencia Artificial

1. Crear un modelo de detección/clasificación de objetos con YOLO
2. Ajustar hiperparámetros.
3. Evaluar el modelo de detección en términos de precisión, sensibilidad y velocidad, generando métricas que se puedan visualizar.
4. Validar el modelo en un conjunto de datos de prueba, identificando errores y ajustando la implementación.

5. Crear un mundo simulado en Webot y controlar un robot para que realice ciertas acciones en base al modelo de yolo.

Big Data Aplicado

2. **Gestión de almacenamiento en S3:**
 - a. Crear y estructurar buckets para datos de entrada y salida.
 - b. Configurar políticas de ciclo de vida en S3 para optimizar el almacenamiento de datos a largo plazo.
3. **Procesamiento y almacenamiento en DynamoDB:**
 - a. Diseño de la tabla/tablas DynamoDB atendiendo a las queries requeridas en la aplicación.
 - b. Usar Lambda con triggers de S3 para cargar automáticamente los datos analizados en DynamoDB
4. **Monitorización del sistema**
 - a. Configurar métricas y logs para monitorizar las operaciones y el rendimiento.
 - b. Usar DynamoDB Streams para capturar cambios en los datos, utilizando una función Lambda que procese estos streams y los registre.
 - c. Crear alertas en Cloudwatch para eventos críticos, como detectar anomalías en el rendimiento o errores en las operaciones.
5. **Optimización de las Read y Write capacity Units**
 - a. Automatizar el cambio de RCU según los datos que se tienen de lecturas por segundo.
 - b. Optimización de las WCU, explicando la toma de decisiones.
6. **Gestión de permisos y roles**
 - a. Identificar los roles y políticas necesarios para que el acceso a los distintos servicios esté controlado y siga el “principio de privilegios mínimos”.

Programación de inteligencia artificial

1. **Diseño de la arquitectura de la solución**, definiendo los componentes clave y su interacción para garantizar un flujo de trabajo eficiente.
2. **Implementación de un proceso de análisis y procesamiento de datos**, asegurando la extracción de información relevante de los archivos proporcionados.
3. **Desarrollo de una interfaz de consulta de datos(via API)**, facilitando el acceso y recuperación de información de manera eficiente
4. **Documentación del proceso y la solución implementada**, proporcionando guías claras para la operación, mantenimiento y posibles mejoras futuras.

Entregables

Arquitectura de la solución (documentación y diagrama) en el Readme del repositorio

- **Descripción de componentes y flujo de trabajo:** con la descripción de los componentes clave, su interacción y el flujo de trabajo del sistema.
- **Diagramas de arquitectura** representando la estructura del sistema y las relaciones entre los componentes.

Código Fuente

- Repositorio de Github estructurado con el código implementado para la extracción y procesamiento de información, incluyendo scripts de procesamiento con ejemplos de ejecución.

- Historial de cambios con ramas organizadas y un control de versiones adecuado para facilitar el desarrollo colaborativo.
- Fichero de despliegue Cloud
- Documento con instrucciones claras de cómo ejecutar la aplicación o despliegue.

Interfaz de consulta de Datos cloud (API)

- **Endpoint definidos que se ajusten al caso de uso específico**

Big Data aplicado

Diseño y Configuración de Almacenamiento en S3:

- **Documento:** Diagrama y explicación de la estructura de buckets (entrada y salida).
- **Capturas de pantalla:** Configuración de los buckets y las políticas de ciclo de vida en AWS S3.

Diseño y Configuración de DynamoDB:

- **Esquema de tabla:** Descripción de la clave primaria, claves secundarias y atributos.
- **Capturas de pantalla:** Configuración de la tabla DynamoDB en AWS.
- **Queries:** Lista de consultas que soporta la tabla con ejemplos en JSON.
- **Explicación:** Breve análisis de cómo el diseño soporta las consultas requeridas. Habrá que hacerlo en la presentación.

Automatización con Lambda y Triggers de S3:

- **Código Lambda:** Archivo **.zip** o **.py** con el código de la función Lambda utilizada.
- **Capturas de pantalla:** Configuración de triggers en S3 y de la integración con DynamoDB.
- **Pruebas:** Evidencia del funcionamiento (logs o capturas de datos cargados en DynamoDB).

Monitorización del sistema:

- **Configuración de métricas:** Capturas de pantalla de métricas y logs en CloudWatch.
- **Evidencia de Streams:** Logs o capturas de pantalla mostrando cómo DynamoDB Streams registra los cambios.
- **Alertas en CloudWatch:** Capturas de las alertas configuradas para eventos críticos.

Optimización de Read y Write Capacity Units:

- **Cálculos:** Justificación de las RCU y WCU iniciales según el patrón de uso.
- **Evidencia:** Capturas de los ajustes realizados en las capacidades de DynamoDB.
- **Automatización:** Código o descripción del proceso para ajustar automáticamente las capacidades.

Gestión de Permisos y Roles:

- **Lista de roles:** Identificación de roles y políticas utilizados en el proyecto.
- **Capturas de pantalla:** Configuración de los roles en IAM.
- **Explicación:** Descripción de cómo los permisos siguen el principio de privilegios mínimos.

MIA:

1. Modelo de detección: Código en GitHub que tiene que incluir el entrenamiento, ajuste y pruebas y validación del modelo. Además de los ajustes de los hiperparámetros y cómo se ha obtenido la mejor solución.
2. Un documento con el siguiente índice:
 - a. Introducción
 - b. Metodología
 - c. Resultados
 - d. Conclusión
3. Una demo del robot (vídeo corto en el que se ve cómo funciona)

SBD:

1. **Infraestructura como Código (IaC):** Archivo docker-compose.yml (o scripts de despliegue) utilizado para levantar los servicios de Kafka y Airflow localmente o en el servidor.
2. **Código de Streaming (Kafka):**
 - a. Script del **Productor** (integrado o separado del modelo de visión).
 - b. Script del **Consumidor** (encargado de leer tópicos y guardar en BBDD).
 - c. Evidencia de funcionamiento: Capturas de consola mostrando mensajes fluyendo en los tópicos o logs del consumidor procesando datos.
3. **Código de Orquestación (Airflow):**
 - a. Archivos Python de los **DAGs** implementados.
 - b. Capturas de la interfaz de Airflow (UI) mostrando el grafo de tareas (Graph View) y ejecuciones exitosas (tiras verdes en Tree/Grid View).
4. **Persistencia:**
 - a. Captura de la tabla de **DynamoDB** con datos reales insertados a través del pipeline.

2. Criterios de evaluación

La evaluación del reto se reflejará en la matriz de evaluación de competencias del mismo.

Criterio	1- Insuficiente	2- Deficiente	3 - Aceptable	4 - Bueno	5 - Excelente
Big Data Aplicado	1 - Insuficiente	2 - Básico	3 - Satisfactorio	4 - Muy Bueno	5 - Excelente
1. Gestión de almacenamiento en S3 (5%)	No se crearon buckets ni políticas de ciclo de vida.	Los buckets están mal estructurados o incompletos. No se configuraron políticas de	Los buckets están estructurados, pero la jerarquía tiene errores menores. Las políticas son básicas o	Los buckets están correctamente estructurados, pero faltan detalles en la jerarquía o la documentación.	Los buckets están bien estructurados, documentados, y las políticas de ciclo de vida optimizan el almacenamiento a largo plazo.

		ciclo de vida.	incompletas.		
2. Procesamiento y almacenamiento en DynamoDB (30%)	La tabla no está diseñada o no responde a las queries requeridas.	La tabla tiene errores significativos o faltan triggers y funciones Lambda funcionales.	La tabla está diseñada para cumplir las queries mínimas, y los triggers y funciones Lambda funcionan parcialmente.	La tabla está bien diseñada para las queries requeridas, y los triggers y funciones Lambda funcionan sin errores.	La tabla está diseñada eficientemente con índices bien justificados, y los triggers y funciones Lambda funcionan perfectamente.
3. Monitorización del sistema (15%)	No se configuraron métricas, logs, Streams ni alertas.	Las métricas y logs son incompletos o no incluyen Streams o alertas funcionales.	Las métricas y logs están configurados, pero los Streams y alertas son básicos o no están optimizados.	Las métricas, logs, Streams y alertas están configurados correctamente pero podrían mejorarse.	Métricas, logs, Streams y alertas están configurados y optimizados para garantizar un monitoreo completo y eficiente.
4. Optimización de Read y Write Capacity Units (10%)	No se realizaron ajustes en las RCU ni WCU.	Se realizaron ajustes básicos, pero no están optimizados ni automatizados.	Las RCU y WCU están configuradas correctamente, pero sin automatización ni una justificación clara.	Las RCU y WCU están bien configuradas con ajustes manuales basados en las necesidades del sistema.	Las RCU y WCU están correctamente configuradas y automatizadas, con una justificación clara de los ajustes.
5. Gestión de permisos y roles (10%)	No se identificaron roles ni políticas necesarias.	Los roles y políticas están incompletos o no cumplen con el principio de privilegios mínimos.	Los roles y políticas cumplen lo básico, pero sin detalles para un control granular.	Los roles y políticas están configurados adecuadamente, siguiendo en gran medida el principio de privilegios mínimos.	Los roles y políticas están correctamente configurados, con una gestión que sigue estrictamente el principio de privilegios mínimos.
Prueba de validación (30%)	Incapaz de realizar la demostración. El alumno no sabe poner en marcha el sistema o desconoce cómo	Defensa confusa e insegura. Logra ejecutar partes del sistema "de memoria", pero falla al explicar el	Defensa correcta y funcional. Demuestra el flujo de datos completo sin errores graves. Explica adecuadamente la función de	Solvente. Demuestra dominio de las herramientas. Puede modificar parámetros en vivo (ej: cambiar un WCU) y mostrar el resultado.	Experto. Defensa brillante. Responde a preguntas de "qué pasaría si..." (escalabilidad, fallos, costes). Demuestra control total sobre la infraestructura

	funcionan las partes integradas.	porqué de las decisiones.	cada componente. Responde dudas estándar.	Justifica las decisiones técnicas.	desplegada y su configuración.
Arquitectura de la solución (documentación y diagrama) en el README del repositorio (10%)					
Arquitectura	1 - Deficiente	2 - Insuficiente	3 - Aceptable	4 - Notable	5 - Excelente
Descripción de componentes y flujo de trabajo	No describe los componentes clave ni su interacción.	Descripción incompleta con información poco clara.	Descripción básica pero funcional.	Descripción clara con algunos detalles menores por mejorar.	Descripción detallada y clara con todos los elementos correctamente explicados.
Diagramas de arquitectura	No se incluyen diagramas.	Diagramas poco claros o incompletos.	Diagramas básicos sin suficiente detalle.	Diagramas claros con leves omisiones.	Diagramas detallados que reflejan fielmente la solución.
Código fuente en Github(40%)					
Código y Git	1 - Deficiente	2 - Deficiente	3 - Aceptable	4 - Notable	5 - Excelente
Código Fuente	Desordenado y sin documentación.	Organización deficiente y confusa.	Organización básica funcional.	Bien estructurado con leves detalles por mejorar.	Organización clara y documentada correctamente.
Control de versiones	No hay historial de cambios.	Historial con pocos commits y sin estructura.	Historial básico con buenas prácticas ocasionales.	Uso adecuado con ramas organizadas.	Gestión eficiente con ramas bien estructuradas y commits descriptivos.
Documentación del despliegue	No se proporciona documentación clara.	Documentación insuficiente para desplegar.	Instrucciones básicas pero comprensibles.	Documentación clara con detalles menores por corregir.	Instrucciones detalladas, probadas y claras.
Interfaz de consulta de datos API (10%)					
Programación de IA	1 - Deficiente	2 - Insuficiente	3 - Aceptable	4 - Notable	5 - Excelente
Funcionalidad de la API	La API no responde correctamente.	Responde parcialmente con errores.	Funcional pero con limitaciones.	Funcional con buena respuesta.	API funcional, eficiente y bien organizada.

Documentación de la API	No se incluye documentación.	Documentación incompleta o confusa.	Documentación básica con errores menores.	Documentación clara con ejemplos prácticos.	Documentación detallada con casos de uso completos.
Prueba de validación (40%)					
SBD	1- Insuficiente	2- Deficiente	3 - Aceptable	4 - Bueno	5 - Excelente
Ingesta y Streaming con Kafka (35%)	No se ha implementado el clúster o no hay transmisión de datos. Los componentes (productor/consumidor) no funcionan.	El clúster funciona pero es inestable. Tópicos mal definidos (mezcla de datos). El productor envía datos pero el consumidor falla o pierde mensajes frecuentemente.	Comunicación funcional: el productor envía JSONs válidos y el consumidor los lee y persiste. Separación básica de tópicos. Estructura de mensaje correcta.	Buena arquitectura de tópicos. El productor maneja bien el envío de datos. El consumidor procesa y persiste sin errores. Código estructurado y limpio.	Arquitectura robusta y escalable. Tópicos optimizados. Gestión avanzada de errores en el consumidor. Manejo eficiente del volumen de mensajes (ej: evita saturación). Documentación técnica impecable.
Orquestación con Airflow (35%)	No se han creado DAGs o estos no ejecutan ninguna tarea. No hay automatización.	DAGs con errores de sintaxis o lógica. Ejecución manual obligatoria. No hay gestión de reintentos ni notificaciones.	DAGs funcionales que ejecutan tareas cíclicas. Automatización básica (por tiempo). Se configuran reintentos simples en las tareas.	Automatización reactiva implementada (ej: detección de archivos). El flujo es lógico y eficiente. Notificaciones de estado configuradas correctamente.	Automatización total y robusta (sensores/eventos). DAGs modulares y mantenibles. Gestión exhaustiva de fallos (reintentos, alertas detalladas). Logs claros y trazabilidad completa del proceso.
Prueba de validación (30%)	Incapaz de realizar la demostración. El alumno no sabe poner en marcha el sistema o desconoce cómo funcionan las partes integradas.	Defensa confusa e insegura. Logra ejecutar partes del sistema "de memoria", pero falla al explicar el porqué de las decisiones.	Defensa correcta y funcional. Demuestra el flujo de datos completo sin errores graves. Explica adecuadamente la función de cada componente. Responde dudas estándar.	Solvente. Demuestra dominio de las herramientas. Puede modificar parámetros en vivo (ej: cambiar un WCU) y mostrar el resultado. Justifica las decisiones técnicas.	Experto. Defensa brillante. Responde a preguntas de "qué pasaría si..." (escalabilidad, fallos, costes). Demuestra control total sobre la infraestructura desplegada y su configuración.

MIA					
Implementación del Modelo-20%	Se ha implementado o correctamente un modelo de detección de objetos con YOLO. Se ha documentado la selección de modelos y su adecuación al problema. Se han ajustado hiperparámetros con pruebas justificadas.	Implementación funcional del modelo de visión con ajustes menores en hiperparámetros. Documentación adecuada de la selección del modelo.	Modelo implementado, pero con errores menores o falta de optimización en hiperparámetros. Documentación con información parcial.	Modelo con errores significativos o sin correcta integración. Documentación insuficiente sobre el ajuste de hiperparámetros.	No se ha implementado correctamente el modelo de visión. No hay documentación.
Evaluación del Modelo-20%	Se ha evaluado el modelo con métricas claras de precisión, sensibilidad y velocidad, acompañadas de visualizaciones comprensibles.	Se han presentado métricas relevantes, aunque sin análisis detallado. Visualizaciones parciales.	Métricas presentadas sin análisis claro. Sin visualizaciones o con información insuficiente.	Métricas inadecuadas o mal interpretadas. Sin visualizaciones o explicaciones.	No se han presentado métricas ni análisis.
Validación del Modelo-20%	Se ha validado el modelo en un conjunto de datos de prueba, identificando errores y justificando cambios en base a métricas y	Se ha realizado validación básica en conjunto de prueba, sin análisis profundo de los errores.	Validación del modelo realizada con pruebas mínimas y sin justificación clara de cambios.	Validación deficiente o inexistente, sin comparación con datos de prueba.	No se ha realizado ninguna validación del modelo.

	pruebas.				
Implementación en Webot-20%	Se ha creado un mundo en Webot funcional con todos los elementos necesarios. Se ha programado un robot capaz de interactuar con su entorno usando un sistema experto. Se ha entregado imagen clara y video demostrando el funcionamiento.	Mundo en Webot funcional con imagen y video entregados. Robot programado con sistema experto básico. Documentación adecuada.	Mundo en Webot con fallos menores. Robot programado sin integración avanzada con el sistema experto. Video sin evidencias claras del comportamiento esperado.	Mundo en Webot incompleto o con errores graves. Robot con problemas funcionales o sin interacción con el entorno. Video deficiente o sin evidencia clara del funcionamiento.	No se ha creado el mundo en Webot. No hay programación de robot ni sistema experto. No se ha entregado imagen, video ni documentación.
Código del Controlador-20%	Se ha implementado o correctamente un controlador. Código estructurado y documentado con comentarios explicativos. Evaluación de distintas configuraciones y su impacto en el rendimiento. Documento detallado con versiones y mejoras.	Controlador funcional con documentación adecuada. Se han probado algunas configuraciones y documentado mejoras, aunque sin análisis profundo. Documento de versiones con cambios aplicados.	Controlador con errores menores o sin optimización. Documentación básica del código y las configuraciones. Documento de versiones con información parcial.	Controlador con errores significativos o funcionamiento deficiente. Falta de documentación clara sobre el código y su funcionamiento. Documento de versiones incompleto o ausente.	No hay implementación funcional del controlador. No hay documentación del código ni registro de cambios. No se ha entregado documento de versiones.

Como procedimiento general de evaluación del reto:

- o Para dar un reto por aprobado en un módulo, la nota del reto deberá ser ≥ 5 (sobre 10) en ese módulo, teniendo en cuenta que hay que tener un mínimo de 2 (sobre 5) en todos los RAs o competencias de ese módulo, y que la nota de las prácticas, exámenes y pruebas de validación (si las hubiera) deberán ser ≥ 2 (sobre 5) para hacer media.
 - o En los retos intermodulares, puede suceder que un alumno/a apruebe el reto en un módulo y lo suspenda en otro; en este caso, realizaría la recuperación sólo del módulo que haya suspendido.
 - o En función de los contenidos de cada módulo, el profesorado considerará realizar o no actividades de validación, como una actividad conceptual/procedimental donde el alumnado demuestre los conocimientos adquiridos en el reto; esta validación será tomada como una evidencia más a la hora de la calificación.
- Si todo el equipo suspende el reto (en 1 o varios módulos), el equipo completo mejorará la documentación y/o presentación del reto, si así lo considera el profesorado.
 - Si sólo parte del equipo suspende el reto, como se terminan los días lectivos, no va a ser posible que el equipo completo colabore para mejorar el reto. En este caso, el profesorado considerará la realización de alguna(s) actividad(es) específica(s) en las que se demuestre, individualmente, los conocimientos adquiridos aplicados al reto; dichas actividades pueden realizarse en común entre varios módulos, o por separado, a criterio del profesorado, y pueden realizarse dentro de las actividades del reto, o con una prueba escrita o con una prueba práctica o con la mejora de la presentación.
 - El alumno/a deberá recuperar la(s) parte(s) que tenga suspendida(s).

Se realiza el feedback con cada equipo, donde se reflejen y se comenten, por una parte, los aspectos en los que el equipo ha funcionado bien, y por otra, se pueda preguntar qué se podría haber hecho para hacerlo mejor.